



Course Title :Generative AI

Unit No : IV

Topics :

Unit-IV Generation of Painting, Music: Variants of GAN, Types of GAN, Cyclic GAN, Using Cyclic GAN to Generate Paintings, Neural Style Transfer, Style Transfer, Music Generating RNN, MuseGAN, Autonomous agents, Deep Q Algorithm, Actor-critic Network.

Mr.M.Balaji
Assistant Professor
Department of CSE(AI&ML)
CMR Technical Campus [Autonomous]
Hyderabad

Variants of GAN

Numerous variants of Generative Adversarial Networks (GANs) have been developed since the original concept was introduced in 2014. While all GANs pit a generator network against a discriminator network, different architectures and objectives allow them to be adapted for specific tasks.

1. DCGAN (Deep Convolutional GAN):

Uses Convolutional and Deconvolutional layers (no fully connected layers)

Employs Batch Normalization for stability.

LeakyReLU in discriminator, ReLU in generator.

Used mainly for image generation.

Example: Generating realistic human faces.

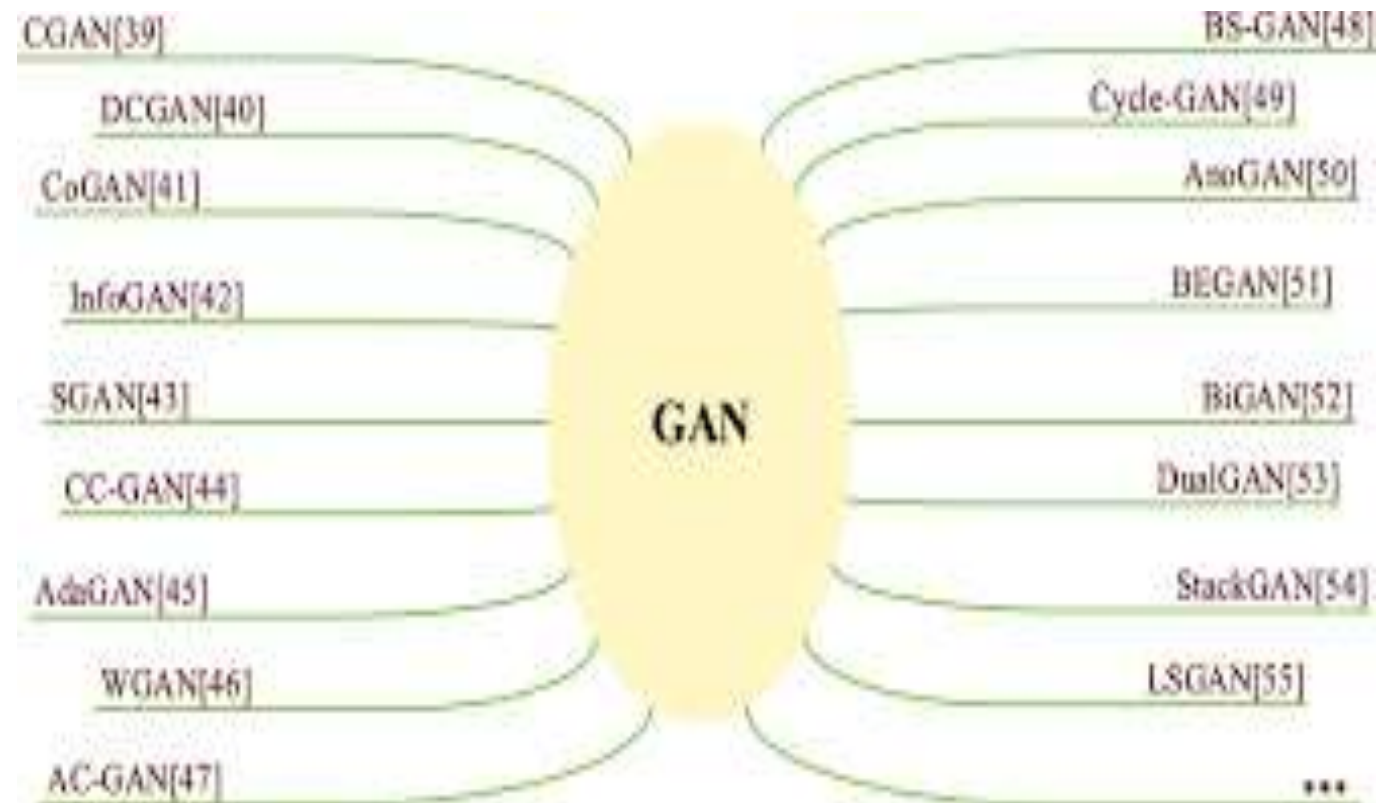
2. CGAN (Conditional GAN):

Adds labels or conditions (e.g., class or text) to both generator and discriminator.

Generates specific category of data.

Helps control the output type.

Example: Generate digit “5” from MNIST dataset when label = 5.



Variants of GAN

3. InfoGAN (Information Maximizing GAN):

Encourages interpretable latent variables.

Maximizes mutual information between latent code and output.

Learns disentangled representations (e.g., rotation, thickness).

Works unsupervised.

4. CycleGAN:

Enables image-to-image translation without paired data.

Uses two generators and two discriminators.

Enforces cycle consistency loss ($A \rightarrow B \rightarrow A$).

Example: Convert horse $\leftrightarrow$ zebra, summer $\leftrightarrow$ winter.

5. Pix2Pix:

Performs **image-to-image translation with paired data**.

Uses **U-Net generator** and **PatchGAN discriminator**.

Example: Sketch $\rightarrow$ photo, black-and-white $\rightarrow$ color.

6. WGAN (Wasserstein GAN):

Uses Wasserstein distance (Earth Mover's distance).

More stable training and less mode collapse.

Requires weight clipping to enforce Lipschitz constraint.

7. WGAN-GP (Wasserstein GAN with Gradient Penalty):

Improves WGAN by using gradient penalty instead of clipping.

More stable and robust training.

Produces high-quality images.

8. LSGAN (Least Squares GAN):

Uses least squares loss instead of binary cross-entropy.

Reduces vanishing gradient problem.

Generates smoother and higher quality images.

Types of GANS

Generative Adversarial Networks (GANs) are a type of deep learning model that use two competing neural networks, a generator and a discriminator, to create new data. Different types of GANs have been developed to address specific challenges and use cases, from generating realistic images to translating images from one domain to another.

Vanilla GAN:

- This is the most basic GAN model, using a simple generator and discriminator to produce data.
- It is known for its training instability, as it is prone to mode collapse, where the generator produces a limited variety of output to "trick" the discriminator.

Deep Convolutional GAN (DCGAN):

- DCGAN was one of the first successful GANs for generating high-quality images, replacing the fully connected layers of earlier models with convolutional neural networks (CNNs).
- It introduced architectural guidelines that improved training stability and led to higher resolution outputs.

Conditional GAN (cGAN)

- A cGAN is a modified GAN that generates data based on specific conditions, such as class labels or text.
- For example, you can tell a cGAN to generate an image of a specific digit, rather than a random one. This "conditioning" allows for more targeted data generation.

Image-to-image translation GANs

CycleGAN

- CycleGAN is designed for image-to-image translation between two unpaired domains, meaning it does not require a one-to-one mapping between training examples.
- For example, it can turn a photo of a horse into a zebra or a summer landscape into a winter scene. It uses a "cycle consistency" loss to ensure that if an image is translated from domain A to B, and then back to A, it returns to the original image.

Pix2Pix

- Unlike CycleGAN, Pix2Pix is a conditional GAN that requires paired image data for training.
- It uses a U-Net architecture to perform "image-to-image translation," such as converting black-and-white photos to color or turning semantic maps into photorealistic images.

High-resolution image GANs

Progressive GAN (ProGAN)

- Developed by NVIDIA, this GAN generates very high-resolution images, up to 1024x1024 pixels.
- It works by starting with low-resolution images and progressively adding layers to both the generator and discriminator as training progresses. This method improves stability and prevents mode collapse.

StyleGAN

- A powerful GAN model also developed by NVIDIA, StyleGAN focuses on generating highly realistic and customizable images, particularly faces.
- Its unique architecture gives users fine-grained control over specific features, such as hair, facial shape, and lighting, across different levels of detail.

Specialized GANs

Super-Resolution GAN (SRGAN)

- SRGAN is trained to upscale low-resolution images to a much higher resolution by filling in realistic details and textures.
- It uses a "perceptual loss" function that focuses on making the high-resolution output perceptually realistic, rather than just minimizing pixel differences.

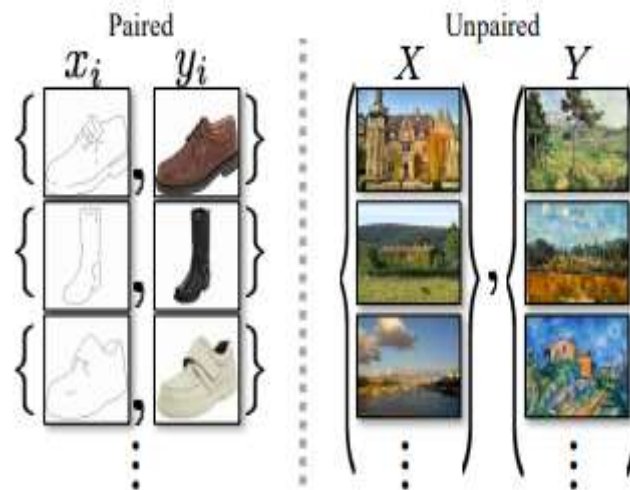
Text-to-Image GAN

- These GANs use a textual description as input to generate a corresponding image.
- Architectures like StackGAN break the task into multiple stages, first generating a low-resolution image from text and then refining it to add more realistic details.

Cyclic GAN

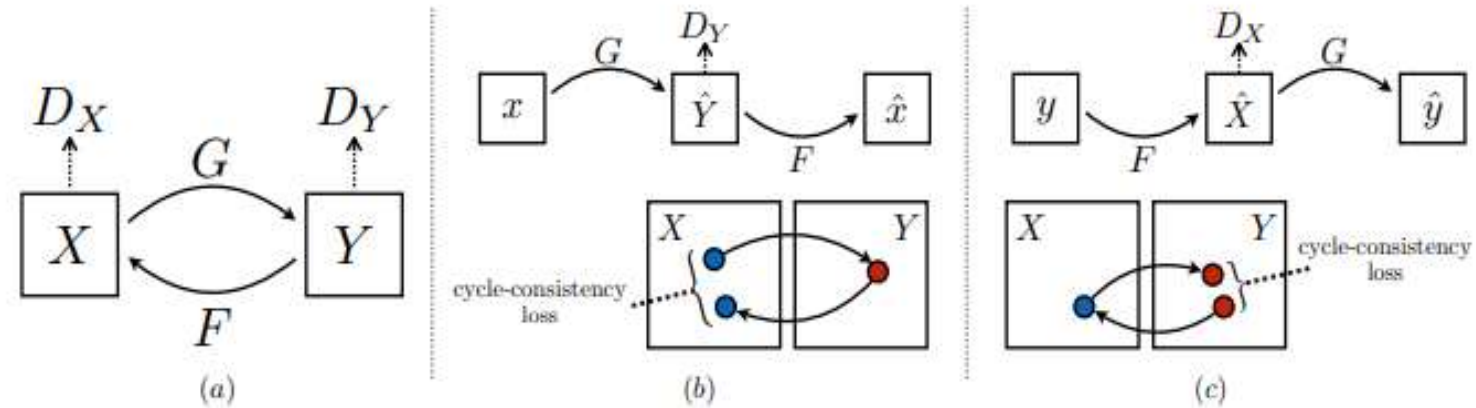
- CycleGAN is a type of GAN used for image-to-image translation without needing paired data. It learns to translate an image from one domain (A) to another domain (B) without requiring exact image pairs.
- CycleGAN solves this problem by learning to change images from one style to another without needing matching pairs. It understands the features of the new style and transforms the original images accordingly.
- This makes it useful for tasks like changing seasons in photos, turning one animal into another or converting pictures into paintings. In this article we will see more about CycleGAN and its core concepts.

- The process starts with an input image(x) and Generator G translates it to the target domain like turning a photo into a painting. Then generator F takes this transformed image and maps it back to the original domain helps in reconstructing an image close to the input.
- The model measures the difference between the original and reconstructed images using a loss function like [mean squared error](#). This cycle consistency loss helps the network to learn meaningful, reversible mappings between the two domains.



Architecture of Cycle GAN

1. Generators: Create new images in the target style.



CycleGAN has two generators G and F :

G transforms images from domain X like photos to domain Y like artwork.

F transforms images from domain Y back to domain X .

The generator mapping functions are as follows:

$$G: X \rightarrow Y$$

$$F: Y \rightarrow X$$

where X is the input image distribution and Y is the desired output distribution such as Van Gogh styles.

2. Discriminators: Decide if images are real (from dataset) or fake (generated).

There are two discriminators D_x and D_y .

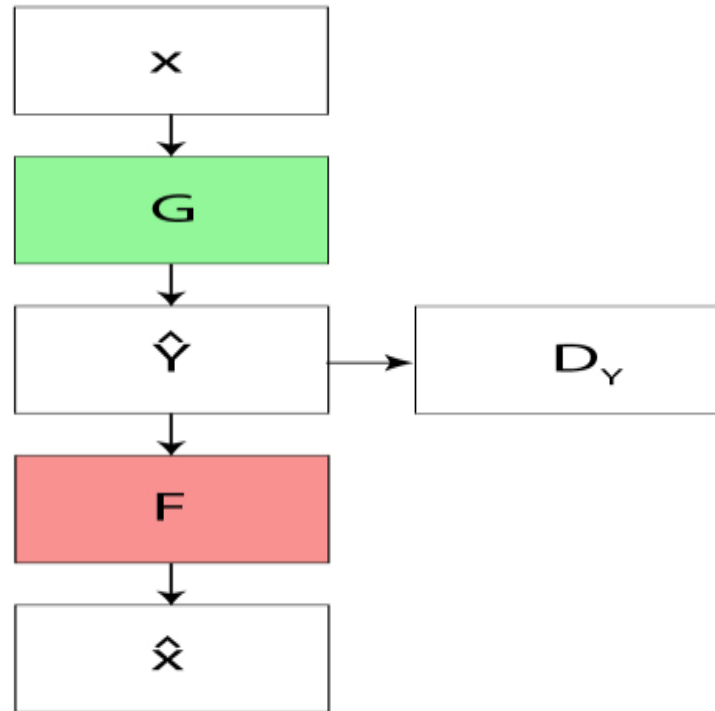
D_x distinguishes between real images from X and generated images from $F(y)$.

D_y distinguishes between real images from Y and generated images from $G(x)$.

To further regularize the mappings the CycleGAN uses two more loss function in addition to adversarial loss.

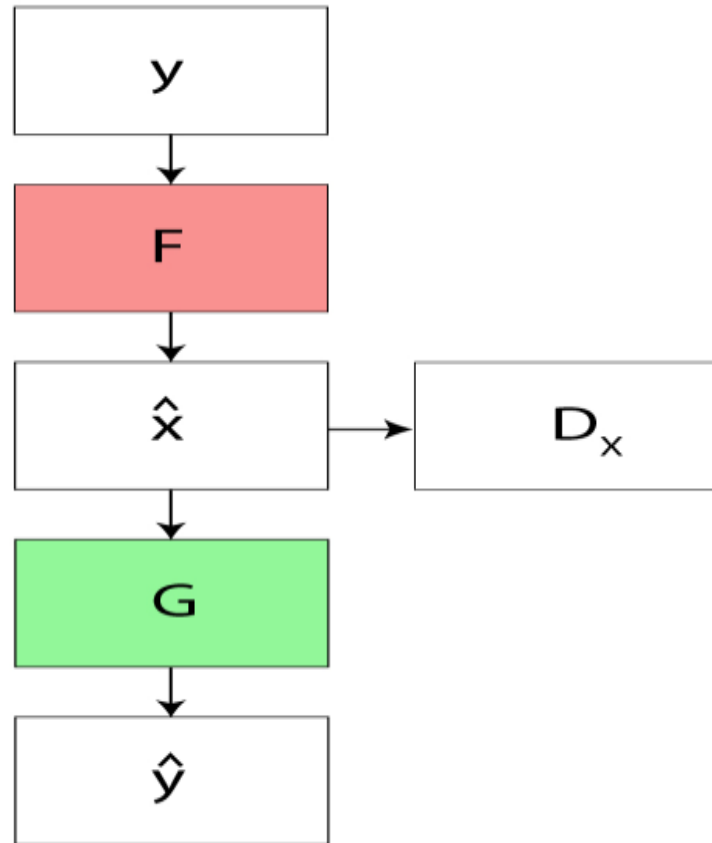
1. Forward Cycle Consistency Loss: Ensures that when we apply G and then F to an image we get back the original image

For example: $x \rightarrow G(x) \rightarrow F(G(x)) \approx x$



2. Backward Cycle Consistency Loss: Ensures that when we apply **F** and then **G** to an image we get back the original image.

For example: $x \xrightarrow{G} G(x) \xrightarrow{F} F(G(x)) \approx x$





Generator Architecture

Each CycleGAN generator has three main sections:

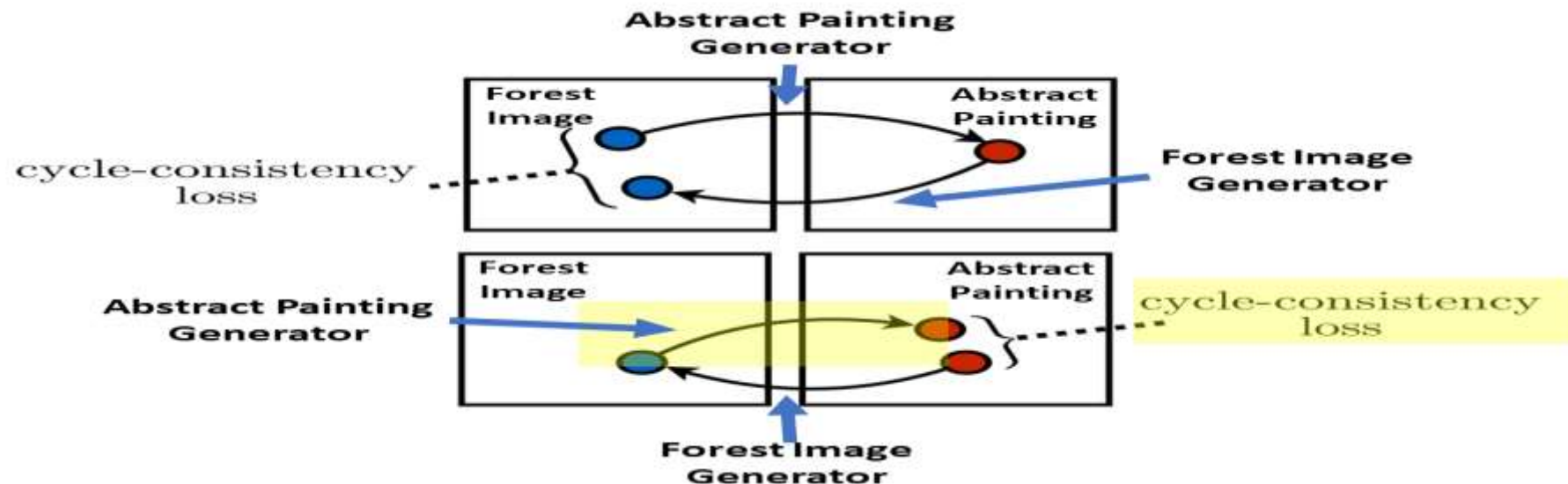
Encoder: The input image is passed through three convolution layers which extract features and compress the image while increasing the number of channels. For example a $256 \times 256 \times 3$ image is reduced to $64 \times 64 \times 256$ after this step.

Transformer: The encoded image is processed through 6 or 9 residual blocks depending on the input size which helps retain important image details.

Decoder: The transformed image is up-sampled using two deconvolution layers and restoring it to its original size.

Using Cyclic GAN to Generate Paintings

- To use Cycle GAN for generating paintings, train two generators and two discriminators on unpaired image datasets of photos and paintings, using a cycle consistency loss to ensure that translating an image from one domain to another and back to the original domain reconstructs the initial image
- This unsupervised approach allows for style transfer, such as converting photographs into paintings of a specific artist or style without needing paired image examples.



1. Model Architecture and Components

Two Generators:

One generator (G_A) learns to convert images from the photo domain to the painting domain, while the other (G_B) learns to convert paintings back to photos.

Two Discriminators:

Each generator has a corresponding discriminator (D_A and D_B) to distinguish between real and generated images within its domain.

Unpaired Datasets:

The model works with two separate datasets: one of photographs and one of paintings (e.g., Impressionist paintings), which do not need to be related.

2. The Cycle Consistency Loss

Forward-Cycle Loss:

An input photo is translated to a painting by G_A , then translated back to a photo by G_B . The cycle consistency loss measures how similar this reconstructed photo is to the original photo.

Backward-Cycle Loss:

The process is reversed, with a painting being translated to a photo by G_B , and then to a painting by G_A . The loss measures the similarity between the final generated painting and the original painting.

Purpose:

By minimizing these cycle losses, the model ensures that the transformations are invertible and that the style is applied correctly without distorting the original image's content.

3. Training Process

Input and Generation:

A photo is fed into G_A to produce a "fake" painting, and a painting is fed into G_B to produce a "fake" photo.

Discriminator Evaluation:

The fake images are then passed to their respective discriminators (D_A for the fake painting, D_B for the fake photo) to assess their authenticity.

Loss Calculation:

Both the adversarial loss (from the discriminators) and the cycle consistency loss are calculated.

Weight Update:

Using backpropagation, the weights of the generators and discriminators are updated to improve their ability to generate realistic images and enforce cycle consistency.

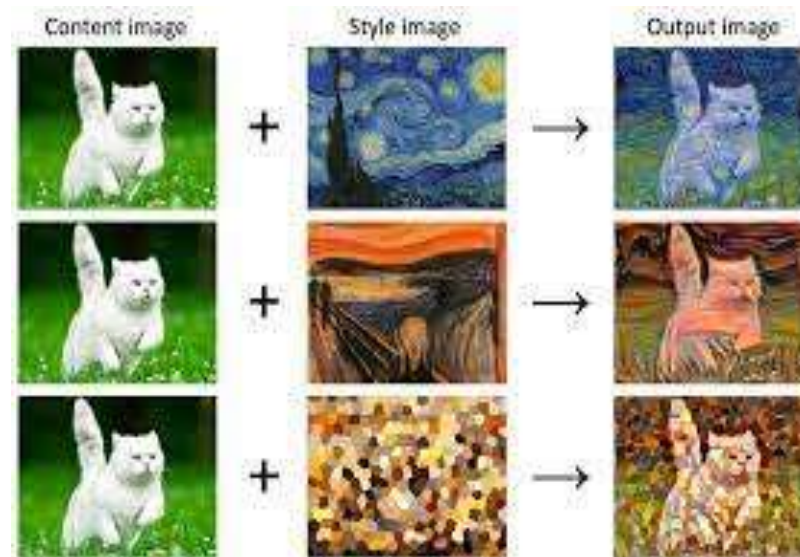
4. Example Application

A CycleGAN can be trained to transform normal photographs into paintings in the style of a famous artist, like [Monet](#), using unpaired photos and Monet's paintings as the datasets.

The output can be a photo styled as a Monet painting, or conversely, a Monet painting stylized to look like a modern photograph.

Neural Style Transfer

- Neural Style Transfer (NST) is a generative AI technique that uses deep learning, primarily convolutional neural networks, to combine the content of one image with the artistic style of another, creating a new, stylized image.
- By treating shallow network layers as extracting style and deep layers as extracting content, NST allows users to reimagine photographs in the manner of famous paintings or apply unique aesthetics to visual content for art, design, and entertainment.



How Neural Style Transfer Works

Input Images:

The process begins with two input images: a content image (the one whose structure you want to preserve) and a style image (the one whose artistic look you want to apply).

Feature Extraction:

A pre-trained [CNN](#) separates the content and style features of these two images.

Content Representation: Deep network layers are used to capture the high-level objects and structure from the content image.

Style Representation: Shallow network layers are used to extract stylistic elements like colors, textures, and brush strokes from the style image.

Generated Image Creation:

A new image is gradually generated by minimizing the difference in content and style between the newly generated image and the content and style images, respectively. This process involves adjusting pixel values to achieve the desired blend.

Applications of NST

- Art and Design:**

- Creating digital art, prototyping visual styles, and generating novel artwork from user-supplied photos.

- Entertainment:**

- Applying consistent visual styles in films and video games and creating unique visual effects.

- Data Augmentation:**

- Improving the robustness of machine learning models by applying diverse styles to a training dataset, making models less sensitive to stylistic variations.

- Marketing:**

- Generating tailored content that resonates with specific audiences by applying different styles to marketing materials.

Style Transfer

Style transfer in generative AI is a process where the artistic style of one image (the style image) is applied to the content of another image (the content image), creating a new image that blends the composition of the content with the visual texture of the style image, often using neural networks like CNNs to achieve this effect.

This technique is widely used in art creation, design, and media for its ability to transform images into unique artistic expressions, such as turning a photograph into a painting or a scene into a comic book style.



Types of Style Transfer

1. Neural Style Transfer (NST)

1. Uses a pretrained CNN (like VGG19).
2. Optimizes the image to mix content and style.
3. Works for single images (slow but high quality).

2. Fast Neural Style Transfer

1. Trains a **feed-forward network** to apply style quickly.
2. Used in real-time applications like filters.

3. GAN-based Style Transfer

1. Uses models like **CycleGAN**, **StyleGAN**, or **StarGAN**.
2. Learns to apply a style to an entire dataset (e.g., photo → Monet painting).
3. Fast and used for large-scale image translation.

How it works

1.Content vs. Style:

The process requires two input images: a content image (the subject or scene you want to preserve) and a style image (the artwork or visual style you want to apply).

2.Neural Networks:

Deep learning models, particularly [Convolutional Neural Networks \(CNNs\)](#), are used to analyze both images.

3.Feature Extraction:

CNNs extract distinct features from each image:

- Content features:** The model identifies the main objects and structure of the content image.
- Style features:** The model captures the texture, color palette, and brushstroke patterns of the style image.

4.Image Synthesis:

A new image is generated that combines the extracted content features with the extracted style features, producing a final artwork that looks like the content image was rendered in the style of the style image.

Examples and applications

- Art Creation:**

Transform photographs into paintings or other artistic forms.

- Film and Animation:**

Convert complex real scenes into animated styles to reduce production costs.

- Design and Marketing:**

Enhance visual content for advertising campaigns, creating stunning and unique images.

- Educational Tools:**

Help users understand how neural networks learn different aspects of images by visually representing pattern recognition.

Music Generating RNN

Recurrent Neural Networks (RNNs) are a foundational component of generative AI for music because their architecture is designed to handle sequential data, such as a sequence of musical notes.

By analyzing a large corpus of music, an RNN can learn the patterns and temporal dependencies within compositions and then generate original pieces of its own.

Step-by-step process:

Input Data (Training):

- MIDI files or symbolic music data (notes, pitch, duration, velocity).

- The data is converted into a **sequence of numerical tokens** representing notes.

Model:

- An **RNN/LSTM network** learns to predict the **next note** given the previous notes.

- The model learns melody, rhythm, and harmony patterns.

Generation:

- Start with a seed (a few notes).

- The model predicts the next note repeatedly to **compose a new piece**.

Output:

- The generated notes are converted back into a **MIDI file**, which can be played using digital instruments.

How RNNs work for music generation

Unlike traditional neural networks that process inputs independently, an RNN has a "memory" that allows it to retain information from previous steps in a sequence. For music, this enables it to produce compositions that maintain coherence over time. The general process involves the following stages:

- **Data preparation:** Musical pieces, usually in MIDI format, are preprocessed to extract note-based features like pitch, duration, and time step. This symbolic representation of music is then converted into a numerical format that the RNN can understand

-

- **Sequential learning:** The RNN is fed sequences of notes from the prepared dataset. For each sequence, the network is trained to predict the next note.

- **Iterative prediction:** Once trained, the model can generate new music by starting with a "seed" sequence of notes. It predicts the next note, adds it to the sequence, and then uses this new, longer sequence to predict the following note. This process repeats to create a new musical piece.

- **Randomness control:** To prevent the model from generating monotonous, repetitive music, a parameter called "temperature" is often used to control the randomness of the predictions. This helps introduce creativity into the composition.

The evolution from RNNs to LSTMs

Traditional RNNs suffer from the vanishing gradient problem, which makes it difficult for them to remember and learn from events in the distant past of a long sequence. This is a major limitation for composing music, which requires maintaining a consistent theme and structure over an extended period.

To overcome this, researchers developed specialized variants of RNNs:

- Long Short-Term Memory (LSTM) networks:** LSTMs use a more complex structure with "gates" that control the flow of information, allowing them to remember or forget information selectively. This capability makes LSTMs particularly effective for handling the long-term dependencies required for generating coherent and structurally sound musical compositions.

- Gated Recurrent Units (GRU) networks:** GRUs are a simplified version of LSTMs that are computationally more efficient. They combine the forget and input gates into a single "update gate" and perform similarly to LSTMs on many tasks.

Hybrid models and modern approaches

To further improve generative AI for music, hybrid models have been developed that combine RNNs with other architectures:

- RNN-VAE**: Models that combine Recurrent Neural Networks with Variational Autoencoders (VAEs) can learn not only the sequential patterns but also a latent representation of musical styles. This allows for more diverse outputs and the ability to control and interpolate between different musical styles.
- Reinforcement learning (RL)**: Some models fine-tune an RNN using reinforcement learning to impose music-theory-based rules as reward functions. This helps the model generate more harmonious and musically pleasing melodies.
- Transformers**: Although RNNs were foundational for music generation, modern generative models often rely on the more advanced transformer architecture. Transformers use a self-attention mechanism that allows them to process long sequences in parallel, more effectively capturing long-range dependencies than RNNs or LSTMs. Examples include Google's Magenta project and OpenAI's MuseNet.

MuseGAN

MuseGAN is a **GAN-based deep learning model** developed to generate **symbolic music** (MIDI format) rather than raw audio.

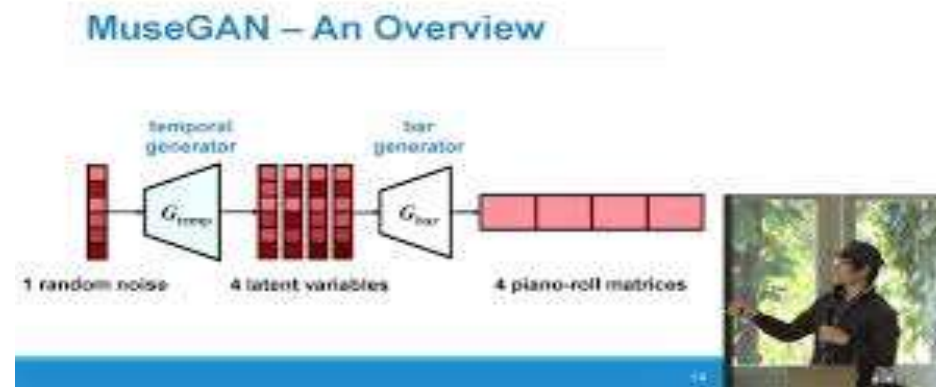
It can generate:

Polyphonic (multiple notes played at once) music

Multi-track (different instruments) compositions

Music that sounds structured, like a short musical phrase or loop

It was proposed by **Hao-Wen Dong, Wen-Yi Hsiao, Li-Chia Yang, and Yi-Hsuan Yang** (2017–2018).



Architecture of MuseGAN

MuseGAN uses **multiple generators and discriminators**, built on the **GAN framework** (Generator + Discriminator).

1. Generator (G)

Produces multi-track piano-roll representations of music.

Can be designed to:

- Generate all tracks together (joint generation)

- Generate each track independently (track-wise generation)

2. Discriminator (D)

Evaluates whether the generated multi-track piano-roll is real (from dataset) or fake (from generator).

Input and Output

•**Input:** Random noise vectors (latent variables) and optionally **conditions** like:

- Chord progressions
- Temporal structures (bars/measures)

•**Output:** Multi-track **piano-rolls**, where:

- Rows = pitches (notes)
- Columns = time steps
- Values = whether a note is active or not

Types of Generators in MuseGAN

MuseGAN allows **different generation schemes**:

Scheme

Description

Unconditional generation

Generates music freely from noise

Inter-track generation

Tracks depend on each other (e.g., drums complement bass)

Intra-track generation

Focuses on coherence within one track

Track-conditional generation

Generates one track conditioned on others

Temporal generation

Generates sequences over time (like multiple bars of music)

Autonomous agents

Autonomous generative AI agents are advanced AI systems that operate with minimal human supervision to achieve complex goals, going beyond basic generative AI by perceiving, reasoning, deciding, and acting independently to complete multi-step tasks.

Unlike chatbots or co-pilots that generate content in response to prompts, these autonomous agents can interact with their environment, learn from outcomes, and adapt their strategies to accomplish objectives over time.

They leverage foundational models, large language models (LLMs), and multimodal AI, and connect to external data sources and systems via APIs to execute tasks across various applications, from business processes to healthcare and transportation.

How Autonomous Agents Work

Perceive the environment:

They gather real-time data from APIs, sensors, databases, or web searches to understand the context of their tasks.

Reason and plan:

Using LLMs and other AI models, they analyze the data to identify patterns, predict outcomes, and formulate a plan to achieve their goals.

Make decisions and take action:

They execute a series of actions, such as sending emails, performing analyses, or controlling physical devices, to reach their objective.

Learn and adapt:

Agents refine their strategies and improve their performance by learning from the outcomes of their actions, a process often enhanced by reinforcement learning.

Key Differences from Traditional Generative AI

Autonomy vs. Prompt-Response:

Traditional generative AI (like text or image generators) responds to direct human prompts, while autonomous agents operate independently, generating their own tasks and working towards a goal without constant human input.

Action-Oriented:

Autonomous agents are designed to act on their environment to complete tasks, whereas generative AI primarily creates new content.

Continuous Learning:

Autonomous agents exhibit a continuous learning loop, adapting and refining their strategies over time, a capability beyond the static nature of many generative models.

Capabilities and Applications

Complex Task Automation:

They automate multi-step business processes and workflows across different functions within an organization.

Proactive Problem-Solving:

They can identify and resolve issues, such as a bookkeeping agent flagging missing invoice data.

Personalized Assistance:

Autonomous agents can provide dynamic, personalized assistance in areas like customer support, adapting to user needs.

Real-World Interaction:

They can interact with various systems and even physical devices, making them suitable for applications like autonomous vehicles or medical robots.

Deep Q Algorithm

To **train an agent** that **generates creative outputs** (text, images, music, etc.) by learning from **rewards or feedback** — instead of labeled data.

Algorithm Steps

1. Initialize

Initialize **Q-network** with random weights θ

$\rightarrow Q(s, a, \theta)$

Initialize **Target Q-network** with same weights $\theta^- = \theta$

Initialize **Replay Memory (D)** to store experiences (s, a, r, s')

2. For each episode (generation round):

Start with an initial state s

(Example: empty canvas, partial text, or musical seed)

Repeat for each step in the episode:

a. **Select an action a** using ε -greedy policy:

With probability ε , choose a random action (exploration)

Otherwise, choose

$$a = \arg \max_a Q(s, a, \theta)$$

)Example: choose next word, pixel, or note)

b. **Perform the action** $\rightarrow$ generate partial or full output.

Get **reward r** based on feedback

(e.g., aesthetic score, human rating, text coherence)

c. **Observe new state s'**

(updated sequence, image, or composition)

d. **Store transition**

(s, a, r, s') in *Replay Memory D*

3.Training the Q-Network

After collecting experiences, train periodically:

Sample a mini-batch of experiences (s, a, r, s') from replay memory D.

Compute target Q-value for each experience:

$$y = r + \gamma \max_{a'} Q(s', a', \theta^-)$$

where

r = reward

γ = discount factor ($0 < \gamma < 1$)

Compute loss:

Update Q-network parameters by minimizing the loss:

$$\theta \leftarrow \theta - \alpha \nabla_{\theta} L(\theta)$$

) α = learning rate)

Periodically update target network:

$$\theta^- \leftarrow \theta .$$

4.End Episode

Repeat until the agent learns to generate high-quality creative outputs

Reduce ϵ gradually (less random exploration over time)

Actor-critic Network

An actor-critic network in generative AI is a Reinforcement Learning (RL) model using two neural networks—an actor for action selection and a critic for evaluating actions—to generate and improve creative output, rather than directly making decisions in a typical RL task.

In this context, the actor generates content (like text or images), and the critic provides feedback by estimating the value of the generated content, guiding the actor to produce better quality, more desirable outputs



How it Works

The Actor:

This neural network proposes and generates content, such as a draft of a story, a draft image, or a Python script for data visualization.

The Critic:

This network evaluates the quality of the content generated by the actor. It functions as a real-time evaluator, similar to a human reviewer.

Learning Process:

The actor learns to produce more effective content by adjusting its generation strategy based on the critic's feedback.

The critic learns to better assess the quality of the generated content, improving its value estimations over time.

Unlike the adversarial relationship in a [Generative Adversarial Network \(GAN\)](#), the actor and critic in this model work collaboratively to achieve a common goal, each improving through the other's feedback.

Examples in Generative AI

Data Visualization:

An Actor-Critic system can generate Python scripts to create data visualizations, with the critic refining these scripts based on the accuracy and visual quality of the resulting graphs.

Robotic Calligraphy:

A GAN's discriminator can be embedded as a critic into an actor-critic model to evaluate and improve a calligraphy robot's ability to write characters by learning from human-like stroke images.

Query Resolution:

A multi-agent system can use an Actor-Critic architecture where an "Actor AI" generates data visualizations to answer user queries, and a "Critic AI" refines these visualizations for clarity and accuracy.

Key Benefits

Improved Quality:

The continuous feedback loop between the actor and critic helps generate more refined and high-quality content.

Flexibility:

The model can handle complex tasks by learning from both the generated actions (content) and their evaluated outcomes.

Efficiency:

It offers a balance between exploration of new creative approaches and exploitation of learned successful patterns.